

Mini coding project 2

Background

I have run a small-scale *in vitro* drug screen: four cell lines (HeLa, MCF7, A549 and HEK293) were treated with two candidate compounds (DrugA and DrugB) at three doses, plus untreated controls, in triplicate. Cell viability and cell counts were recorded for each sample. You have been provided with two dataframes:

- `cell_line_info` which describes the four cell lines used in the experiment
- `cell_viability` which is the main dataframe with the experimental results in it

If you are unable to see these in your R environment, please ask a staff member for assistance.

All questions in the assignment require the tidyverse package, so please load it using the code chunk below:

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

Part A: exploring data (8 marks)

Question 1 (3 marks total)

Write R code to explore the `cell_viability` dataset:

a. What is the average cell count?

Solution.

You need to use the `mean()` function on the `cell_count` column. There are multiple ways for you to do this, but no matter what you must include the `na.rm` argument (or remove NAs in another way, e.g. `na.omit()`). Two example solutions are provided below:

```
# using $ and na.rm = TRUE
mean(cell_viability$cell_count, na.rm = TRUE)
```

```
[1] 35575.44
```

```
# using pull() and na.omit()
cell_viability %>% pull(cell_count) %>% mean(na.rm = TRUE)
```

```
[1] 35575.44
```

b. What are the three lowest viability percentages?

Solution.

You can use either the `slice_min()` function or the `arrange()` function and read off the three lowest values:

```
# using slice_min()
cell_viability %>% slice_min(viability_percent, n = 3)
```

```
# A tibble: 3 x 7
  sample_id cell_line treatment dose_uM replicate viability_percent cell_count
  <chr>      <chr>      <chr>    <int>    <int>         <dbl>         <dbl>
1 S065      MCF7        DrugB     100        2           5.9           968
2 S066      MCF7        DrugB     100        3           6.9           692
3 S020      HeLa        DrugA     100        2           7.7          5204
```

```
# using arrange()
cell_viability %>% arrange(viability_percent) %>% head(n = 3)
```

```
# A tibble: 3 x 7
  sample_id cell_line treatment dose_uM replicate viability_percent cell_count
  <chr>      <chr>      <chr>      <int>      <int>          <dbl>      <dbl>
1 S065      MCF7      DrugB        100         2           5.9        968
2 S066      MCF7      DrugB        100         3           6.9        692
3 S020      HeLa      DrugA        100         2           7.7       5204
```

The three lowest viability percentages are 5.9%, 6.9% and 7.7%

c. How many samples had a viability above 50% ?

Solution.

Use the `filter()` function, with a logical test on the `viability_percent` column. You can then either just read off the number of rows from the tibble printout or use the `nrow()` function to count them:

```
# using filter
cell_viability %>%
  filter(viability_percent > 50) %>%
  # use nrow() to count how many
  nrow()
```

```
[1] 53
```

53 of the samples have a viability above 50%

Question 2 (3 marks total)

I would like to investigate the `cell_viability` data with respect to the cell lines. **Grouping the data based on cell line:**

a. How many samples are there in each cell line? (1 mark)

Solution.

You can use `summarise()` with the `n()` helper function:

```
cell_viability %>%
  summarise(
    count = n(),
    .by = cell_line
  )
```

```
# A tibble: 4 x 2
  cell_line count
  <chr>      <int>
1 MCF7        21
2 HEK293      21
3 HeLa        21
4 A549        21
```

It is also possible to just use the `count()` function (fun fact: this is basically just a helper that internally runs the grouped summary for you)

```
cell_viability %>% count(cell_line)
```

```
# A tibble: 4 x 2
  cell_line    n
  <chr>      <int>
1 A549        21
2 HEK293      21
3 HeLa        21
4 MCF7        21
```

Either way, there are 21 samples per cell line.

- b. Calculate the average cell count for each cell line (1 mark)

Solution.

With the grouping from part a), take the mean:

```
cell_viability %>%
  summarise(
    count = n(),
    av_cell_count = mean(cell_count, na.rm = TRUE),
    .by = cell_line
  )
```

```
# A tibble: 4 x 3
  cell_line count av_cell_count
  <chr>      <int>      <dbl>
1 MCF7        21      24864.
2 HEK293       21      46637.
3 HeLa        21      36136.
4 A549        21      33188.
```

Don't forget `na.rm = TRUE` or `na.omit()` to remove the missing values. Note that I left the `count = n()` in from the previous question, but this is not required to get the mark.

c. Now, group by cell line and drug treatment (1 mark)

Solution.

Remember, to group by two columns you need to put them in a vector when using `.by`:

```
cell_viability %>%
  summarise(
    count = n(),
    av_cell_count = mean(cell_count, na.rm = TRUE),
    .by = c(cell_line, treatment)
  )
```

```
# A tibble: 12 x 4
  cell_line treatment count av_cell_count
  <chr>      <chr>      <int>      <dbl>
1 MCF7      DrugB         9      19637.
2 HEK293    DrugA         9      43853.
3 HeLa      DrugB         9      33422.
4 HeLa      DrugA         9      32574.
5 A549      DrugA         9      32646.
6 HEK293    Control        3      59035.
7 HeLa      Control        3      52871.
8 MCF7      DrugA         9      25929.
9 HEK293    DrugB         9      45289.
10 A549     DrugB         9      29459.
11 MCF7     Control        3      38368.
12 A549     Control        3      45997.
```

Whereas, if you use `group_by()`, they just both need to go into the function as arguments:

```
cell_viability %>%
  group_by(cell_line, treatment) %>%
  summarise(
    count = n(),
    av_cell_count = mean(cell_count, na.rm = TRUE)
  )
```

`summarise()` has regrouped the output.
 i Summaries were computed grouped by cell_line and treatment.
 i Output is grouped by cell_line.
 i Use `summarise(.groups = "drop\_last")` to silence this message.
 i Use `summarise(.by = c(cell\_line, treatment))` for per-operation grouping
 (`?dplyr::dplyr\_by`) instead.

```
# A tibble: 12 x 4
# Groups:   cell_line [4]
  cell_line treatment count av_cell_count
  <chr>      <chr>    <int>      <dbl>
1 A549      Control      3      45997.
2 A549      DrugA         9      32646.
3 A549      DrugB         9      29459.
4 HEK293    Control      3      59035.
5 HEK293    DrugA         9      43853.
6 HEK293    DrugB         9      45289.
7 HeLa      Control      3      52871.
8 HeLa      DrugA         9      32574.
9 HeLa      DrugB         9      33422.
10 MCF7     Control      3      38368.
11 MCF7     DrugA         9      25929.
12 MCF7     DrugB         9      19637.
```

Note that this output is grouped.

Question 3 (2 marks)

There is one more dataset that we'd like to work with, the 'plate_reader_export.csv' file in the 'data' folder.

Write code to read this file into R, and don't forget to save it as an appropriately named variable!

Solution.

Read in the data using `read_delim()` or `read_csv()`, and assign the result to a variable:

```
plate_reader_export <- read_csv("data/plate_reader_export.csv")
```

Rows: 12 Columns: 7

-- Column specification -----

Delimiter: ","

chr (2): cell_line, treatment

dbl (5): replicate, dose_0, dose_1, dose_10, dose_100

i Use ``spec()`` to retrieve the full column specification for this data.

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

As always, you may use the RStudio GUI, as long as the code is copied into the .qmd file.

Part B: cleaning/wrangling data (12 marks)

Question 4 (2 marks)

The `cell_viability` dataset has some missing values.

- Use code to remove the missing values in the `viability_percent` column (1 mark)

Solution.

You can remove the missing values using `filter()` and the `is.na()` helper function like so:

```
cell_viability %>%  
  # don't forget to negate is.na with !  
  # otherwise you'll get only the na values  
  filter(!is.na(viability_percent))
```

A tibble: 77 x 7

	sample_id	cell_line	treatment	dose_uM	replicate	viability_percent	cell_count
	<chr>	<chr>	<chr>	<int>	<int>	<dbl>	<dbl>
1	S066	MCF7	DrugB	100	3	6.9	692
2	S041	HEK293	DrugA	1	2	100	59007
3	S053	HeLa	DrugB	10	2	78.2	42703

```

4 S021      HeLa      DrugA      100      3      14      8492
5 S031      A549      DrugA       1      1      92.5     42993
6 S055      HeLa      DrugB     100      1      21.9     9446
7 S063      MCF7      DrugB     10      3      68.2      NA
8 S013      HeLa      DrugA       1      1      96.3     52432
9 S033      A549      DrugA       1      3      100      48190
10 S018      HeLa      DrugA     10      3      73.6     42163
# i 67 more rows

```

Seeing as the question didn't specifically ask to remove only missing values in the viability_percent column, using `na.omit()` is also acceptable:

```
na.omit(cell_viability)
```

```

# A tibble: 72 x 7
  sample_id cell_line treatment dose_uM replicate viability_percent cell_count
  <chr>      <chr>      <chr>    <int>    <int>      <dbl>      <dbl>
1 S066      MCF7      DrugB     100      3         6.9        692
2 S041      HEK293     DrugA       1      2        100       59007
3 S053      HeLa      DrugB     10      2        78.2      42703
4 S021      HeLa      DrugA     100      3         14        8492
5 S031      A549      DrugA       1      1        92.5      42993
6 S055      HeLa      DrugB     100      1        21.9      9446
7 S013      HeLa      DrugA       1      1        96.3      52432
8 S033      A549      DrugA       1      3        100      48190
9 S018      HeLa      DrugA     10      3        73.6      42163
10 S012      HEK293     Control    0      3        100      59768
# i 62 more rows

```

Note that this will remove 5 additional rows, which contain NA in other columns.

- b. I'd like to replace the NA values in `viability_percent` and `cell_count` with something to indicate that all the cells died. I tried to write the following code, but got an error. What is the problem and how could I fix it? (1 mark)

```

cell_viability %>%
  replace_na(
    list(viability_percent = "dead",
         cell_count = "dead")
  )

```

Error in `vec\_assign()`:

! Can't convert `replace\$viability\_percent` <character> to match type of `data\$viability_per

[Solution.](#)

The problem here is that I'm trying to replace missing numbers (of class 'double') with a character type. Instead, I should use a number, like 0 for example

Question 5 (2 marks)

I want to create a wide-format summary table showing the average viability for each combination of cell line and treatment, so I can know which cell line is the most drug-resistant.

The table should have a column with the cell lines, and a column for each treatment, with the cells showing the average viability like so:

cell_line	Control	DrugA	DrugB
A549	...	...	...
HEK293	...	...	...
HeLa	...	...	...
MCF7	...	...	...

Write R code to create this table. Hint: you first need to find the average viability for each cell line/treatment combination with a grouped summarise(), then use pivot_wider() to construct the table.

[Solution.](#)

As the hint describes, do the grouped summary first, and then use pivot wider:

```
cell_viability %>%
  summarise(
    # don't forget to remove missing values somehow
    av_viability = mean(viability_percent, na.rm = TRUE),
    .by = c(cell_line, treatment)
  ) %>%
  # then, pivot_wider
  pivot_wider(
    # names_from is the new column names, which in the example
    # table above come from the 'treatment' column
    names_from = treatment,
    # values_from is what values belong in the table, so the
```

```
# average viability. NOT the 'viability_percent' because
# the question asked for average viability for each combo
values_from = av_viability
)
```

```
# A tibble: 4 x 4
  cell_line DrugB DrugA Control
  <chr>      <dbl> <dbl>   <dbl>
1 MCF7      57.5  59.8    98.7
2 HEK293     74   70.4    99.1
3 HeLa      65.1  47.0    97.4
4 A549      62.2  69.2    98.6
```

Question 6 (2 marks total)

In Question 3, you read in 'plate_reader_export.csv', which is an example of the raw drug screening data that was used to make the cell_viability dataframe we've been working with.

Your task is to wrangle plate_reader_export from a wide format into a long format.

- What do you think a long-form version of this data would look like (i.e. what columns might it have)? (1 mark)

Solution.

A long-form version would contain a column called 'dose' (or similar), which contains the different drug dosages (0, 1, 10, 100 etc), rather than each drug dosage being in its own column. This is how the cell_viability dataframe is currently set up.

- Write code to create the long-form version of this dataframe (1 mark)

Solution.

Use pivot_longer() as follows:

```
plate_reader_export %>%
  pivot_longer(
    # cols are the columns to pivot from
    # use starts_with() to select all of dose_0, dose_1 etc
    cols = starts_with("dose_"),
    # names_to is the name of the new column that contains
```

```

# all the old column names. it needs to be in quotes
# as this column doesn't exist yet
names_to = "dose_uM",
# values_to is the name of the new column that contains
# all of the values of the columns described in 'cols'
# again, this needs to be in quotes
values_to = "viability_percent"
)

```

```

# A tibble: 48 x 5
  cell_line treatment replicate dose_uM viability_percent
  <chr>      <chr>      <dbl> <chr>      <dbl>
1 HeLa      DrugA          1 dose_0      98.6
2 HeLa      DrugA          1 dose_1      96.3
3 HeLa      DrugA          1 dose_10     79.7
4 HeLa      DrugA          1 dose_100    10.5
5 HeLa      DrugA          2 dose_0      99.5
6 HeLa      DrugA          2 dose_1     100
7 HeLa      DrugA          2 dose_10     69.1
8 HeLa      DrugA          2 dose_100     7.7
9 HeLa      DrugA          3 dose_0      94.1
10 HeLa     DrugA          3 dose_1      99.7
# i 38 more rows

```

Question 7 (3 marks total)

I would like to add the cell_line_info dataframe into the cell_viability dataframe.

- Write code to join these two dataframes (1 mark)

Solution.

You can use `left_join()` as follows:

```
left_join(cell_viability, cell_line_info)
```

Joining with ``by = join_by(cell_line)``

```
# A tibble: 84 x 10
  sample_id cell_line treatment dose_uM replicate viability_percent cell_count
  <chr>      <chr>      <chr>      <int>      <int>      <dbl>      <dbl>
1 S066      MCF7      DrugB       100         3         6.9        692
2 S041      HEK293    DrugA        1         2        100       59007
3 S053      HeLa      DrugB        10         2        78.2      42703
4 S021      HeLa      DrugA       100         3         14        8492
5 S031      A549      DrugA        1         1        92.5      42993
6 S055      HeLa      DrugB       100         1        21.9      9446
7 S063      MCF7      DrugB        10         3        68.2        NA
8 S013      HeLa      DrugA        1         1        96.3      52432
9 S033      A549      DrugA        1         3        100      48190
10 S018      HeLa      DrugA        10         3        73.6      42163
# i 74 more rows
# i 3 more variables: tissue_origin <chr>, cancer_type <chr>,
#   doubling_time_hours <int>
```

```
tibble(left_join(cell_viability, cell_line_info)) == tibble(left_join(cell_line_info, cell_viability))
```

- b. Is there a difference between the result of joining `cell_line_info` to `cell_viability` or doing it the opposite way (`cell_viability` to `cell_line_info`)? In other words, does it matter which order we join the dataframes in? (1 mark)

Solution.

The only difference is the order in which the columns appear (whichever dataframe is first, those columns appear first), i.e. `left_join(cell_viability, cell_line_info)` puts the columns from `cell_viability` first, whereas `left_join(cell_line_info, cell_viability)` puts the columns from `cell_line_info` first.

But ultimately, it doesn't matter which order we join them in, as the information in the output dataframe is still the same

- c. Imagine that `cell_line_info` was missing the HEK293 cell line. In this case, would it matter which order we join the dataframes in? (1 mark)

Solution.

Yes, in this case it does matter! That's because, `left_join()` keeps all the rows in the left (first) dataframe, so when not every row has a match, the order of dataframes dictates whether non-matching rows will be silently dropped or kept with missing values in the newly added columns.

Let's demonstrate:

```
# first, create a cell_line_info without HEK293
info_no_hek293 <- cell_line_info %>% filter(cell_line != "HEK293")

# join them with cell_viability first
left_join(cell_viability, info_no_hek293)
```

Joining with `by = join\_by(cell\_line)`

```
# A tibble: 84 x 10
  sample_id cell_line treatment dose_uM replicate viability_percent cell_count
  <chr>      <chr>      <chr>      <int>      <int>      <dbl>      <dbl>
1 S066      MCF7      DrugB        100         3         6.9         692
2 S041      HEK293     DrugA         1         2        100        59007
3 S053      HeLa      DrugB         10         2        78.2       42703
4 S021      HeLa      DrugA        100         3         14         8492
5 S031      A549      DrugA         1         1        92.5       42993
6 S055      HeLa      DrugB        100         1        21.9       9446
7 S063      MCF7      DrugB         10         3        68.2         NA
8 S013      HeLa      DrugA         1         1        96.3       52432
9 S033      A549      DrugA         1         3        100       48190
10 S018      HeLa      DrugA        10         3        73.6       42163
# i 74 more rows
# i 3 more variables: tissue_origin <chr>, cancer_type <chr>,
#   doubling_time_hours <int>
```

```
# it has 84 rows: HEK293 rows from cell_viability are kept,
# and NA is put in the newly added columns
```

```
# join with info_no_hek293 first
left_join(info_no_hek293, cell_viability)
```

Joining with `by = join\_by(cell\_line)`

```
# A tibble: 63 x 10
  cell_line tissue_origin cancer_type doubling_time_hours sample_id treatment
  <chr>      <chr>      <chr>      <int>      <chr>      <chr>
1 HeLa      Cervix      Cervical ade~        24 S053      DrugB
2 HeLa      Cervix      Cervical ade~        24 S021      DrugA
3 HeLa      Cervix      Cervical ade~        24 S055      DrugB
4 HeLa      Cervix      Cervical ade~        24 S013      DrugA
```

```

5 HeLa      Cervix      Cervical ade~      24 S018      DrugA
6 HeLa      Cervix      Cervical ade~      24 S016      DrugA
7 HeLa      Cervix      Cervical ade~      24 S052      DrugB
8 HeLa      Cervix      Cervical ade~      24 S003      Control
9 HeLa      Cervix      Cervical ade~      24 S019      DrugA
10 HeLa     Cervix      Cervical ade~      24 S014      DrugA
# i 53 more rows
# i 4 more variables: dose_uM <int>, replicate <int>, viability_percent <dbl>,
#   cell_count <dbl>

```

```
# it has 63 rows: HEK293 rows from cell_viability are dropped
```

This highlights the importance of thinking about your joins and the order you do them in!

Question 8 (3 marks)

My colleague has already started analysing the data, and sent me the following code:

```

dose_summary <- cell_viability %>%
  filter(treatment != "Control") %>%
  summarise(
    av_viability = mean(viability_percent, na.rm = TRUE),
    .by = c(cell_line, treatment, dose_uM))

```

- a. Describe in words what this code is doing (1 mark)

Solution.

This code first removes the control samples, then calculates the average viability for each cell line/drug/dosage combination (i.e. collapses all of the replicates) and finally assigns this dataframe to the `dose_summary` variable

- b. I'm working on creating a powerpoint presentation, and I want to include `dose_summary` as a table on my slide. How can I write out `dose_summary` as a **CSV file**? (1 mark)

Solution.

You can write a CSV file using the function `write_csv()`, which needs two arguments: the variable to write out (in this case `dose_summary` and then a file path to save it as)

```
write_csv(dose_summary, "dose_summary.csv")
```

- c. I want to highlight in my presentation the % viability of HEK293 cells at a dose of 100 uM in Drug B. Write R code to extract this value out of dose_summary. (1 mark)

Solution.

Here, you just need to use filter. You can combine the conditions with & or just use separate filter functions:

```
# using &
dose_summary %>%
  filter(treatment == "DrugB" & cell_line == "HEK293" & dose_uM == 100)
```

```
# A tibble: 1 x 4
  cell_line treatment dose_uM av_viability
  <chr>      <chr>      <int>      <dbl>
1 HEK293    DrugB          100         41.1
```

```
# using multiple filter() functions
dose_summary %>%
  filter(treatment == "DrugB") %>%
  filter(cell_line == "HEK293") %>%
  filter(dose_uM == 100)
```

```
# A tibble: 1 x 4
  cell_line treatment dose_uM av_viability
  <chr>      <chr>      <int>      <dbl>
1 HEK293    DrugB          100         41.1
```